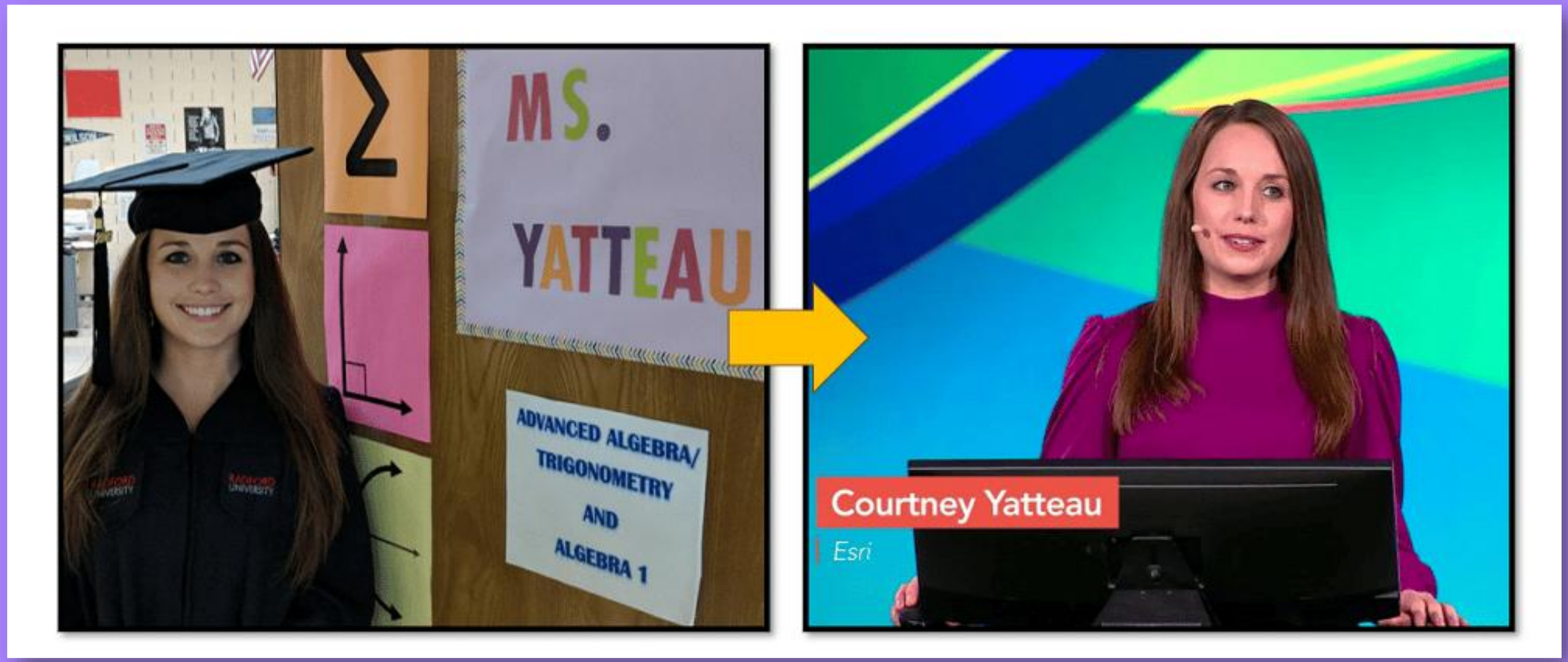

REACT AND THE ART OF GAMIFICATION
COURTNEY YATTEAU
START

COURTNEY YATTEAU

Developer Advocate, Esri



AGENDA

01

INTRO DEMO: ONE TINY FEEDBACK LOOP

02

WHAT GAMIFICATION ACTUALLY MEANS

03

MAIN DEMO: QUEST APP

04

G.A.M.E.S. IN REACT

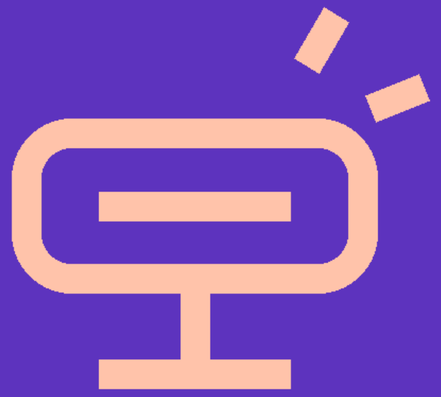
05

INTRO DEMO CALLBACK (IF TIME)

06

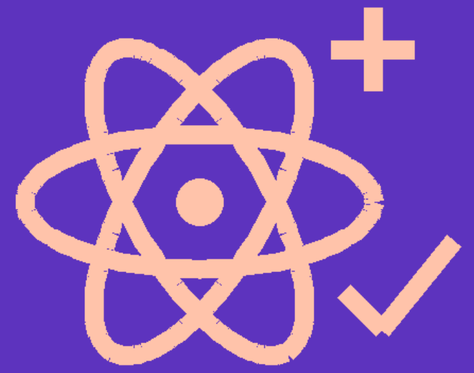
EXTENSIONS AND TAKEAWAYS

BEFORE THEORY: FEEL THE LOOP



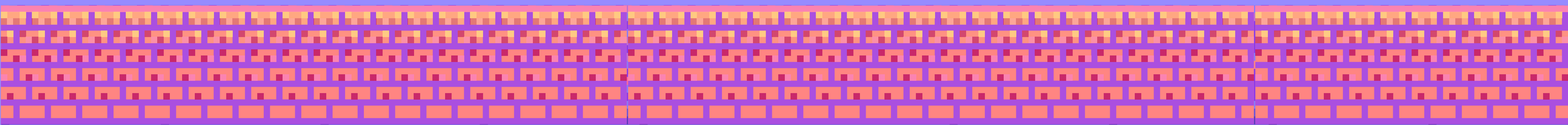
One button

One delayed reward



One tiny React upgrade

INTRO DEMO: INSTANT XP



WHAT CHANGED?

BEFORE

Click → wait → reward

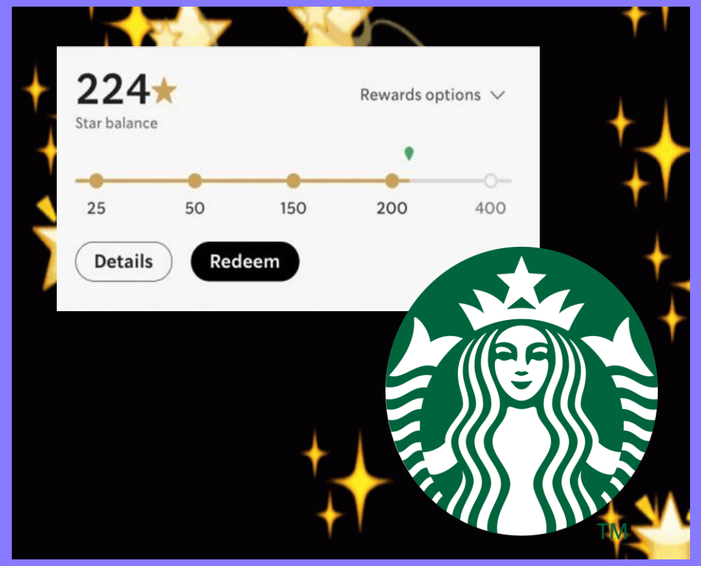
AFTER

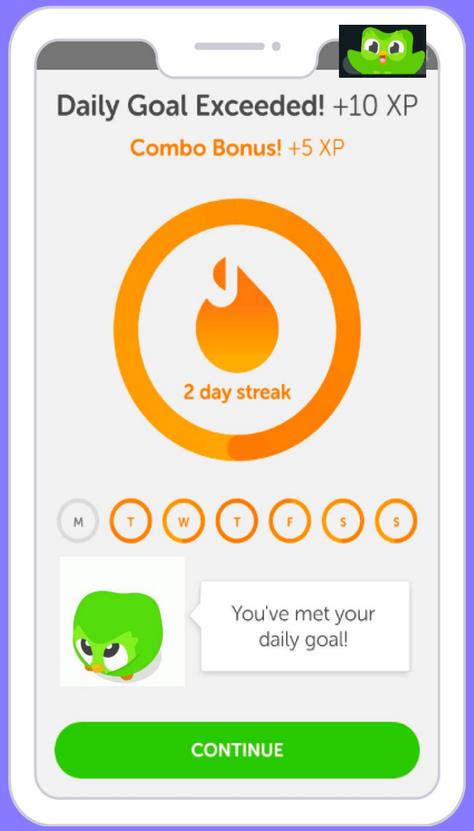
Click → instant feedback → save → confirm

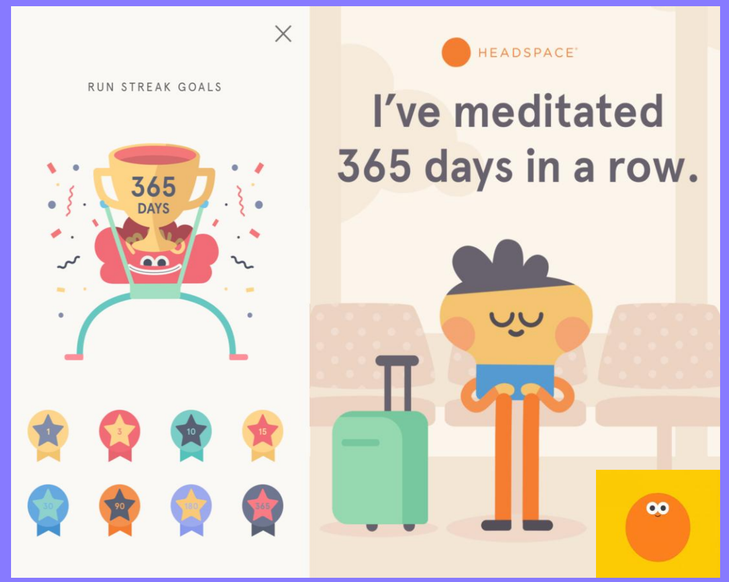
WHAT IS GAMIFICATION?

> Gamification is the process of using **game thinking** and **game dynamics** to **engage audiences** and **solve problems**.
>
> Gabe Zichermann

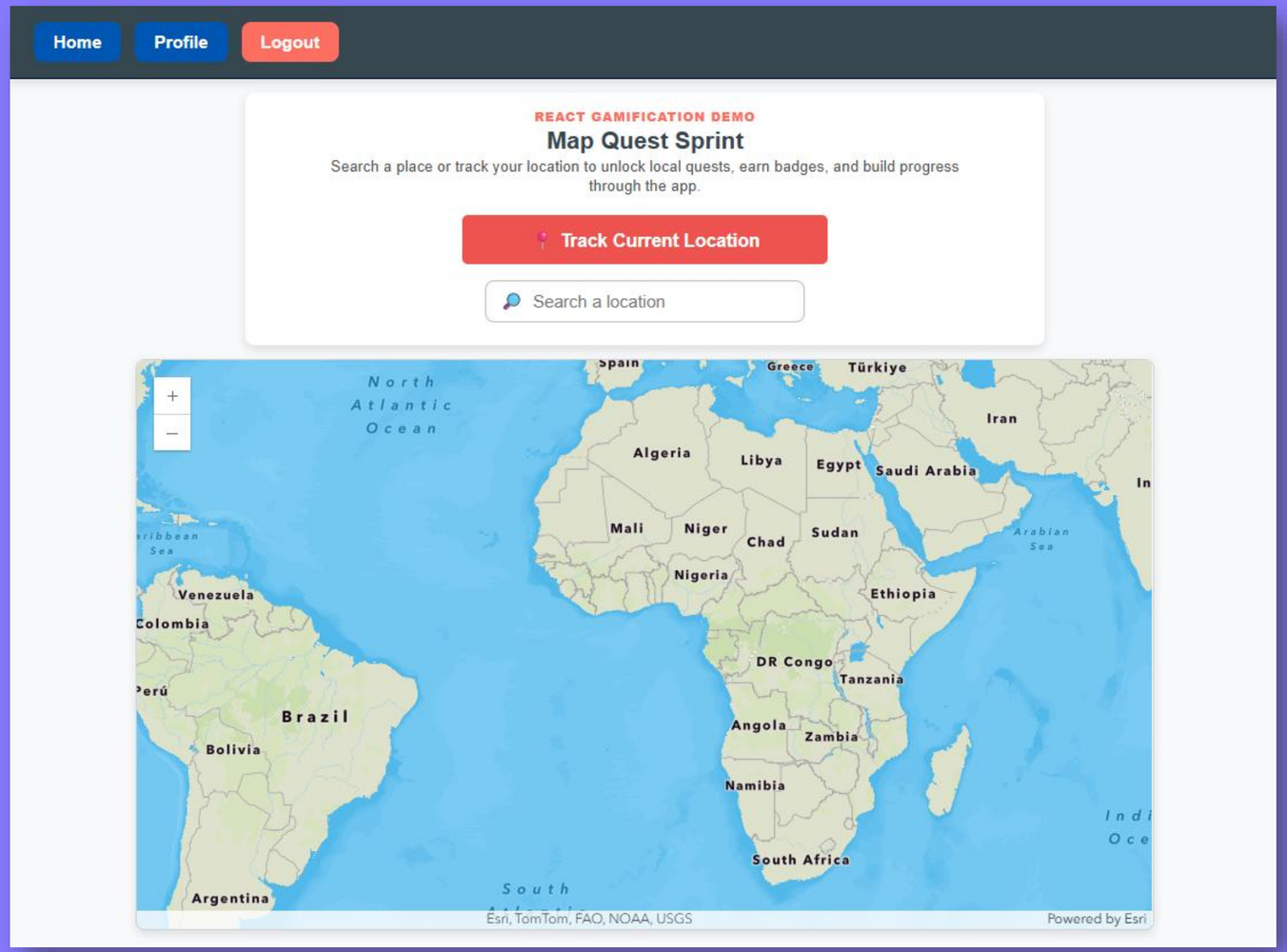
WHY GAMIFICATION?



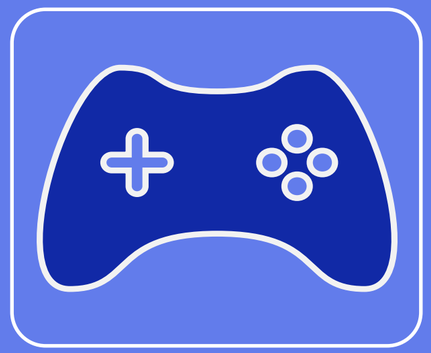




FROM ONE LOOP TO A FULL EXPERIENCE



G.A.M.E.S.



G

Gamified UI Elements

A

M

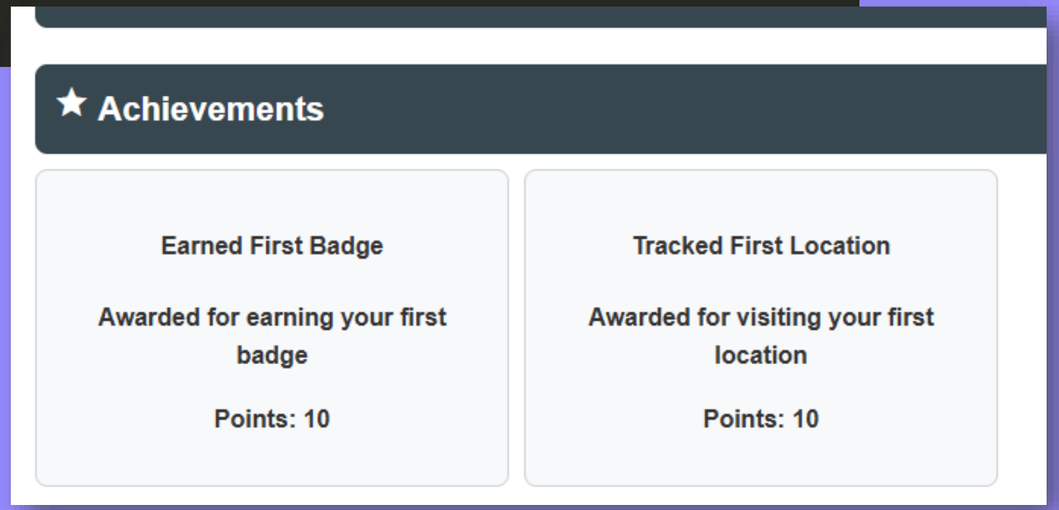
E

S

Gamified UI Elements – Achievements

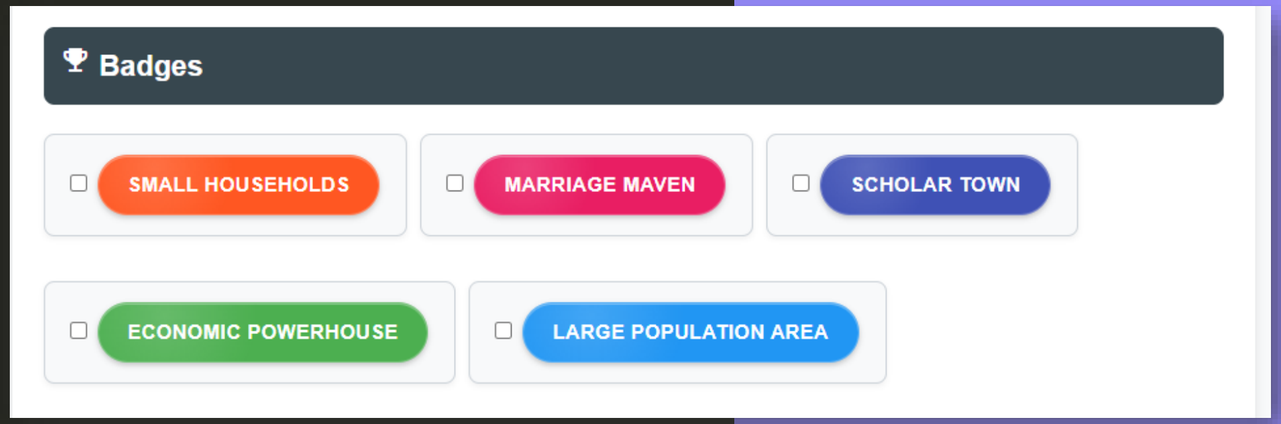
```jsx
const showAchievementPopupMessage = (points, text) => {
  setAchievementMessage(`Congrats! You've earned ${points} points for ${text}`);
  setShowAchievementPopup(true);
  setTimeout(() => {
    setShowAchievementPopup(false);
  }, 3000);
};
```

```jsx
{earnedAchievements.map((achievement, index) => (
  <div key={index} className="achievement-card">
    <h4>{achievement.text}</h4>
    <p>{achievement.description}</p>
    <p>Points: {achievement.points}</p>
  </div>
))}
```



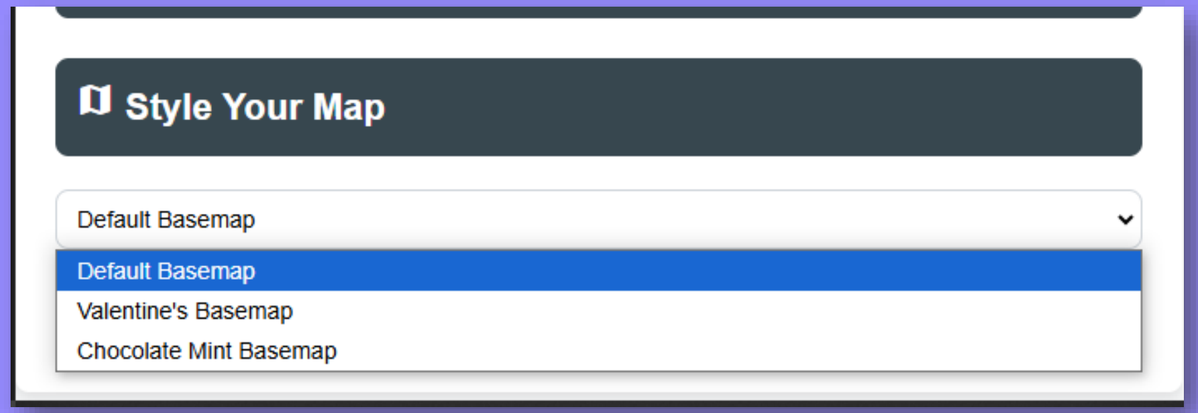
Gamified UI Elements – Badge

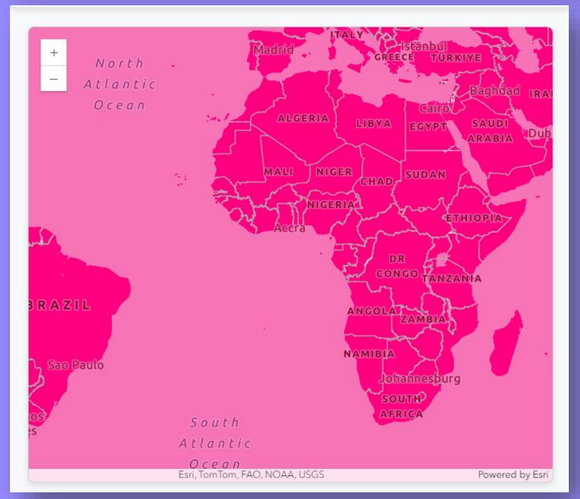
```jsx
const Badge = ({ badges }) => {
  const [earnedBadges, setEarnedBadges] = useState([]);
  const [selectedBadge, setSelectedBadge] = useState(null);
  const handleBadgeClick = (badge) => {
    setSelectedBadge(badge);
    setEarnedBadges((prev) => [...prev, badge]);
  };
  return (
    <div className="badges-container">
      {badges.map((badge, index) => (
        <div
          key={index}
          className={`badge ${earnedBadges.includes(badge) ? "badge-earned" : ""}`}
          style={{ backgroundColor: badge.color }}
          onClick={() => handleBadgeClick(badge)}
        >
          {badge.text}
        </div>
      ))}
      {selectedBadge && <BadgePopup badge={selectedBadge} onClose={() => setSelectedBadge(null)} />}
    </div>
  );
};
```

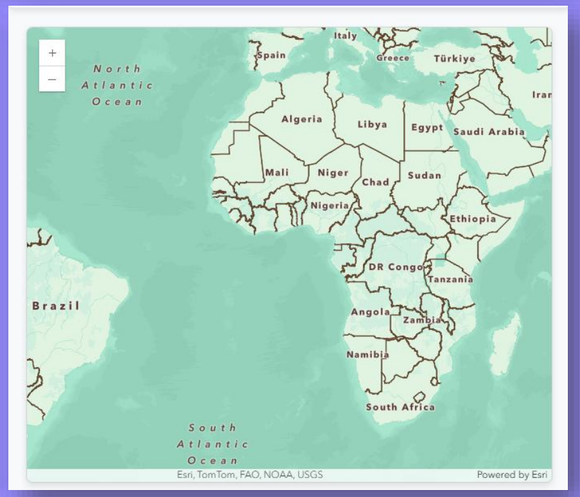


Gamified UI Elements – Basemap Options

```
const basemapOptions = [
  {
    id: "11b7300674584eb793129a808290d235",
    name: "Default Basemap",
    unlocked: true,
  },
  {
    id: "456d1df3810e482b8abcb2aa0440d6ac",
    name: "Valentine's Basemap",
    unlocked: earnedAchievements.length >= 1,
  },
  {
    id: "f030ad3c601c4c4f9404197ded54b8e6",
    name: "Chocolate Mint Basemap",
    unlocked: earnedAchievements.length >= 2,
  },
],
```





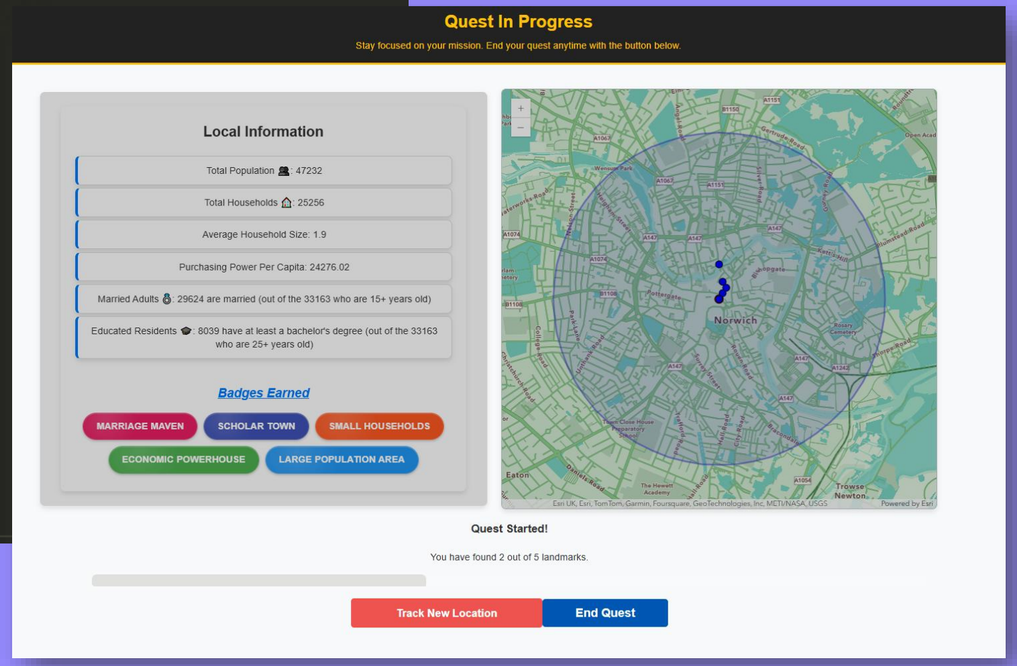


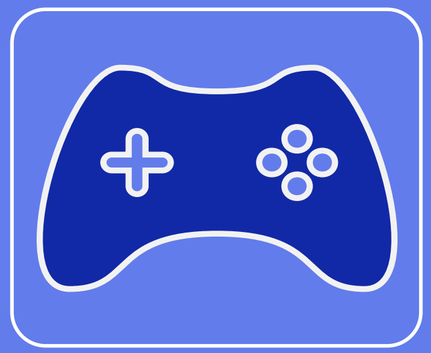
Gamified UI Elements – Progress Bar

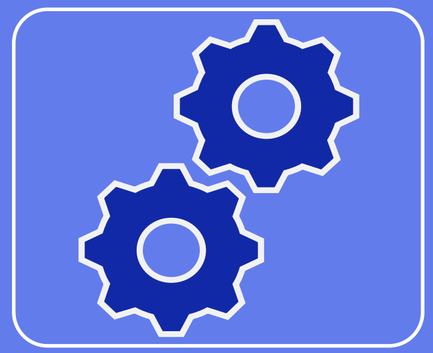
```jsx
const ProgressBar = ({ points, maxPoints }) => {
  const progress = (points / maxPoints) * 100;
  return (
    <div>
      <div style={{ width: `${progress}%` }} />
      <span>{points} / {maxPoints} Points</span>
    </div>
  );
};

export default ProgressBar;
```

Gamified UI Elements – Quests

```jsx
const QuestStatus = ({ questStarted, foundLandmarks, totalLandmarks }) => {
  if (!questStarted) return null;
  const progressPercentage = Math.round((foundLandmarks / totalLandmarks) * 100);
  return (
    <div className="quest-status-container">
      <h3>Quest Started!</h3>
      <p>
        You have found {foundLandmarks} out of {totalLandmarks} landmarks.
      </p>
      <div className="progress-bar-container">
        <div
          className="progress-bar"
          style={{ width: `${progressPercentage}%` }}
        ></div>
      </div>
    </div>
  );
};
```







G	A	M	E	S
Gamified UI Elements	**A**dvanced State Control			

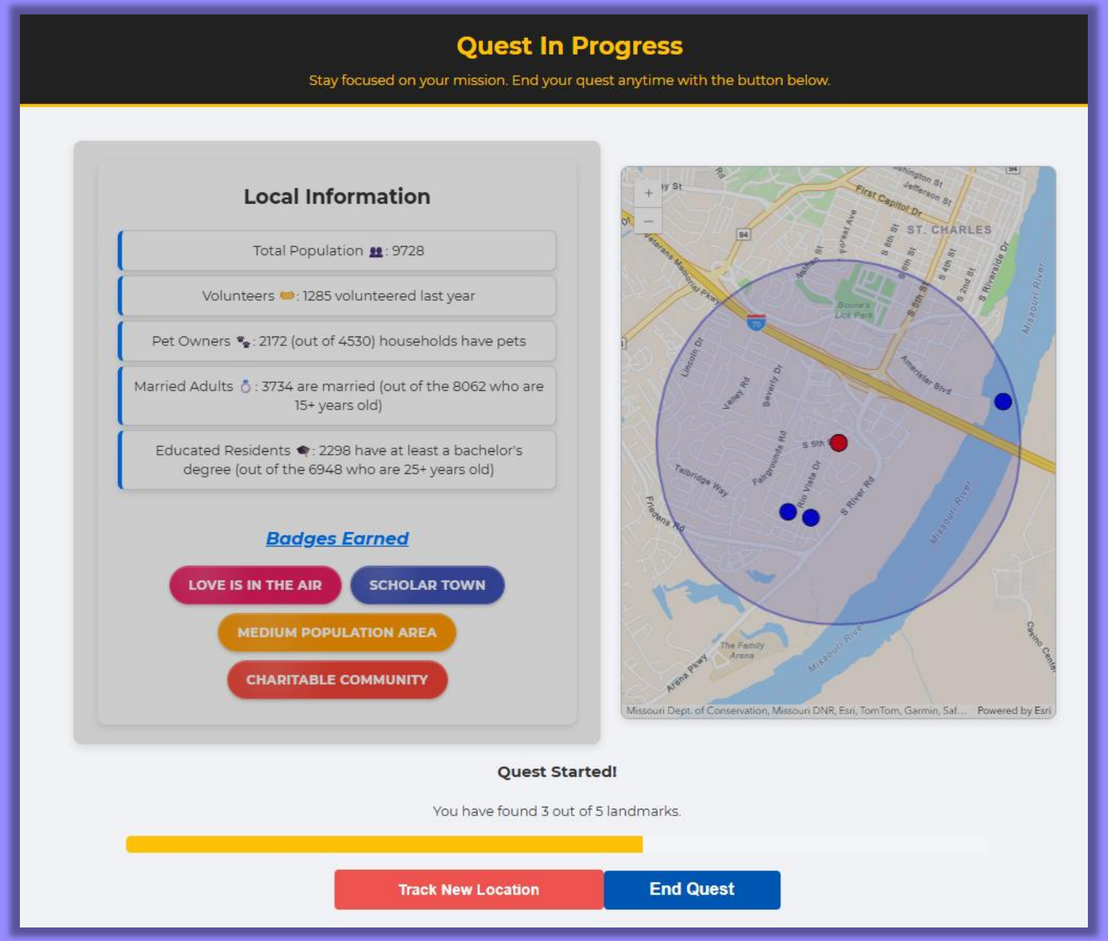
Advanced State Control – Context and Reducer

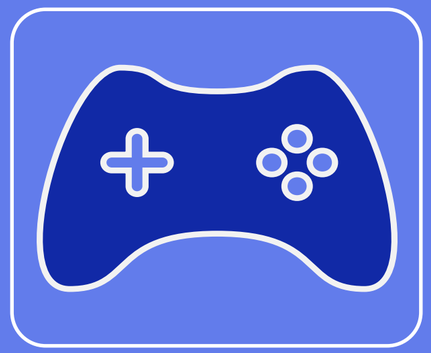
```jsx
import { createContext, useReducer, useContext } from "react";

const AppContext = createContext();

const initialState = {···
};

const reducer = (state, action) => {
  switch (action.type) {···
  }
};

export const AppProvider = ({ children }) => {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <AppContext.Provider value={{ state, dispatch }}>
      {children}
    </AppContext.Provider>
  );
};

export const useAppContext = () => useContext(AppContext);
```

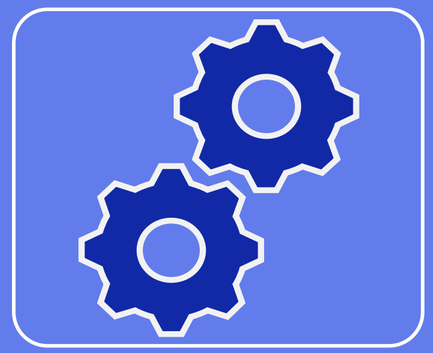
```jsx
switch (action.type) {
  case "SET_LOCATION":
    return { ...state, location: action.payload };
  case "SET_LOCATION_INPUT":
    return { ...state, locationInput: action.payload };
  case "SET_SUBMITTED":
    return { ...state, submitted: action.payload };
  case "RESET":
    return initialState;
  case "SET_SELECTED_BADGE":
    return { ...state, selectedBadge: action.payload };
  case "SET_SELECTED_BASEMAP":
    return { ...state, selectedBasemap: action.payload };
  case "EARN_BADGE":
    return {
      ...state,
      badges: [...state.badges, action.payload],
    };
  default:
    throw new Error(`Unhandled action type: ${action.type}`);
}
```

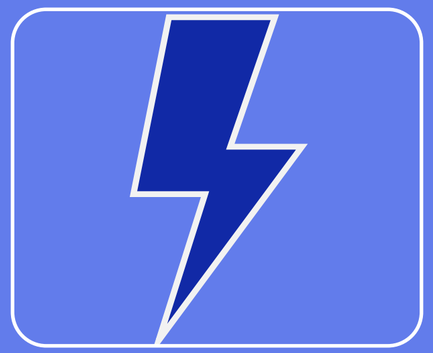
Advanced State Control – Quest View

```jsx
<QuestStatus
  questStarted={questStarted}
  foundLandmarks={foundLandmarks.length}
  totalLandmarks={nearbyLandmarks.length}
/>
{questStarted && (
  <div className="quest-buttons">
    <button onClick={handleTrackNewLocation}>
      Track New Location
    </button>
    <button onClick={handleEndQuestButtonClick}>
      End Quest
    </button>
  </div>
)}
```







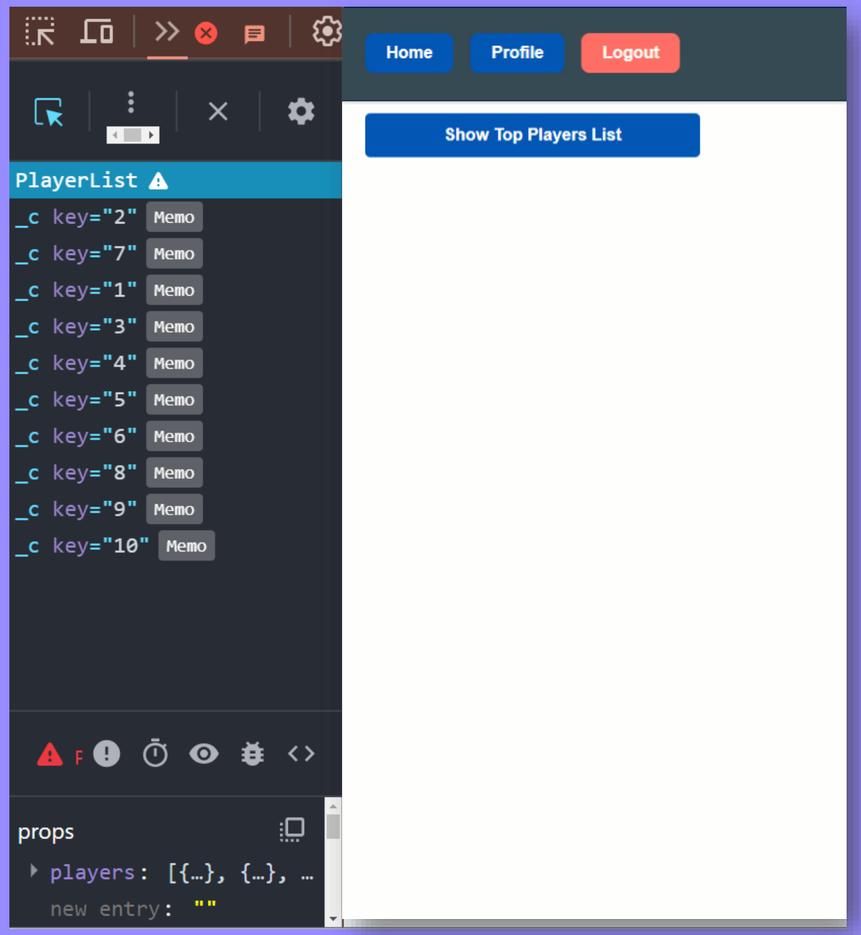


G	A	M	E	S
Gamified UI Elements	**A**dvanced State Control	**M**emoization /Modern Optimization		

Memoization – Players List

```jsx
import React from 'react';

const Player = React.memo(({ player }) => {
  console.log(`Rendering ${player.name}`);
  return (
    <li>
      {player.name}: {player.score}
    </li>
  );
});

export default Player;
```

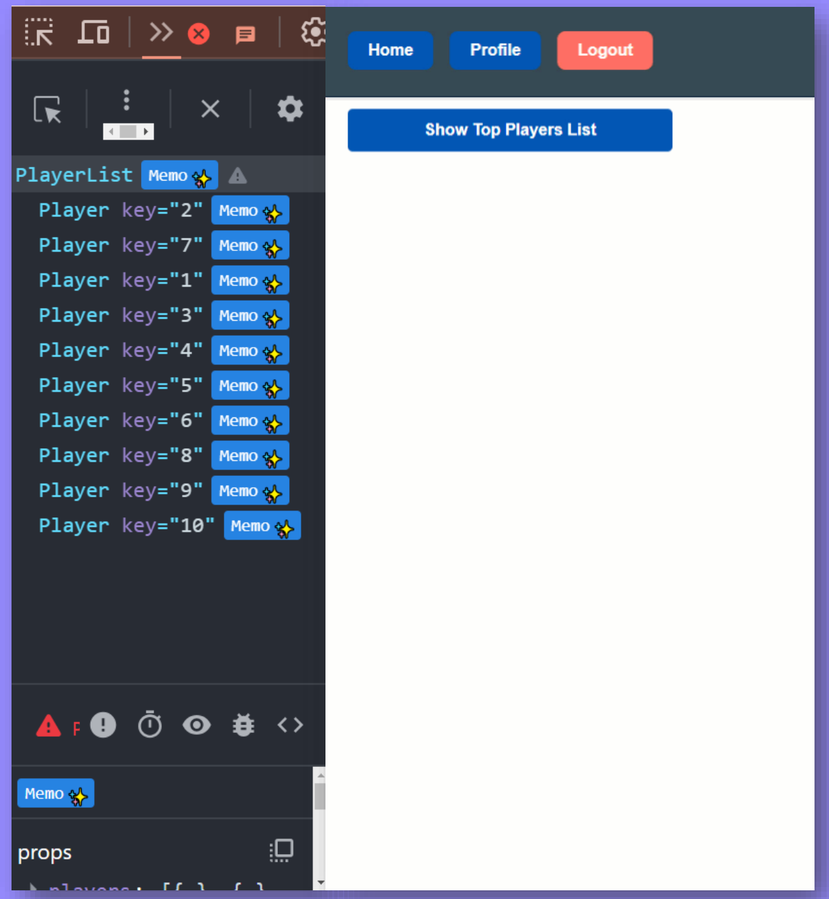
```jsx
import { useMemo } from 'react';
import Player from './Player';

const PlayerList = ({ players }) => {
  const sortedPlayers = useMemo(() => {
    return players.sort((a, b) => b.score - a.score);
  }, [players]);

  return (
    <div className="player-list">
      <h3>Top Players</h3>
      <ul>
        {sortedPlayers.map(player => (
          <Player key={player.id} player={player} />
        ))}
      </ul>
    </div>
  );
};

export default PlayerList;
```



Memoization – Players List with React Compiler

```jsx
const Player = React.memo(({ player }) => {
  console.log(`Rendering ${player.name}`);
  return (
    <li>
```

```jsx
import { useMemo } from 'react';
import Player from './Player';

const PlayerList = ({ players }) => {
  const sortedPlayers = useMemo(() => {
    return players.sort((a, b) => b.score - a.score);
  }, [players]);
```

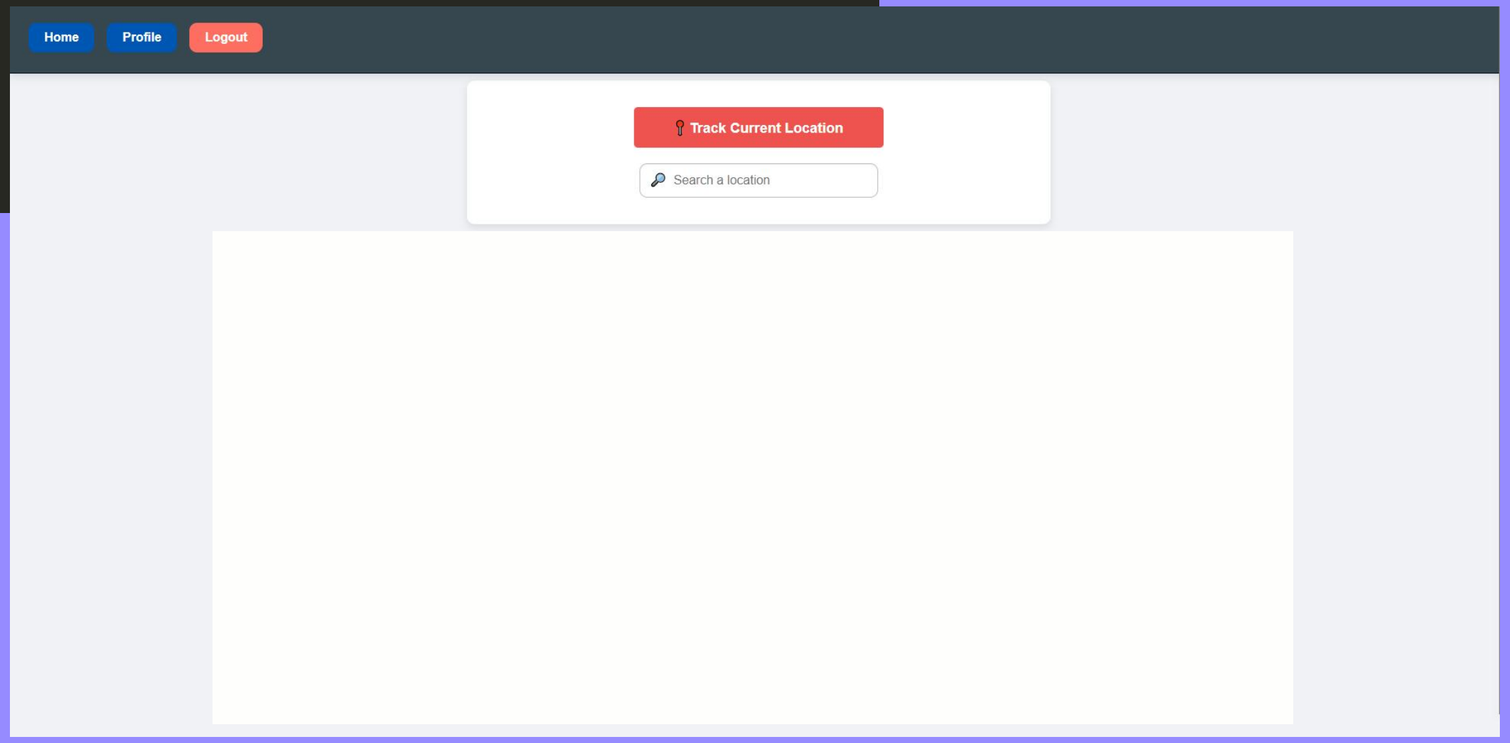


G	A	M	E	S
Gamified UI Elements	**A**dvanced State Control	**M**emoization /Modern Optimization	**E**fficient Rendering	

Efficient Rendering – Lazy and Suspense

```
const MapViewComponent = lazy(() => import("./components/MapViewComponent"));
const SimpleMapComponent = lazy(() => import("./components/SimpleMapComponent"));
```

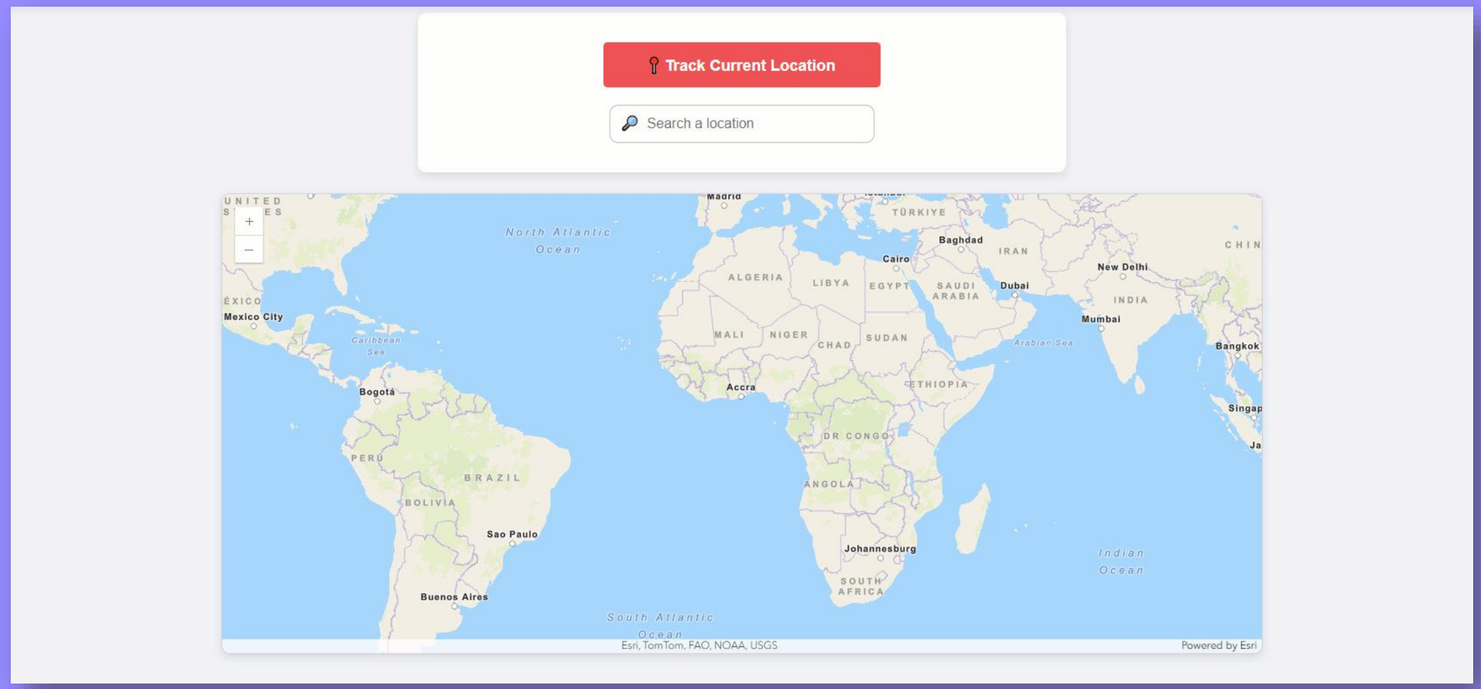
```
<Suspense fallback={<div>Loading Map...</div>}>
  {state.location ? (
    <MapViewComponent location={state.location} landmarks={nearbyLandmarks} />
  ) : (
    <SimpleMapComponent  />
  )}
</Suspense>
```



Efficient Rendering – useEffect and useRef

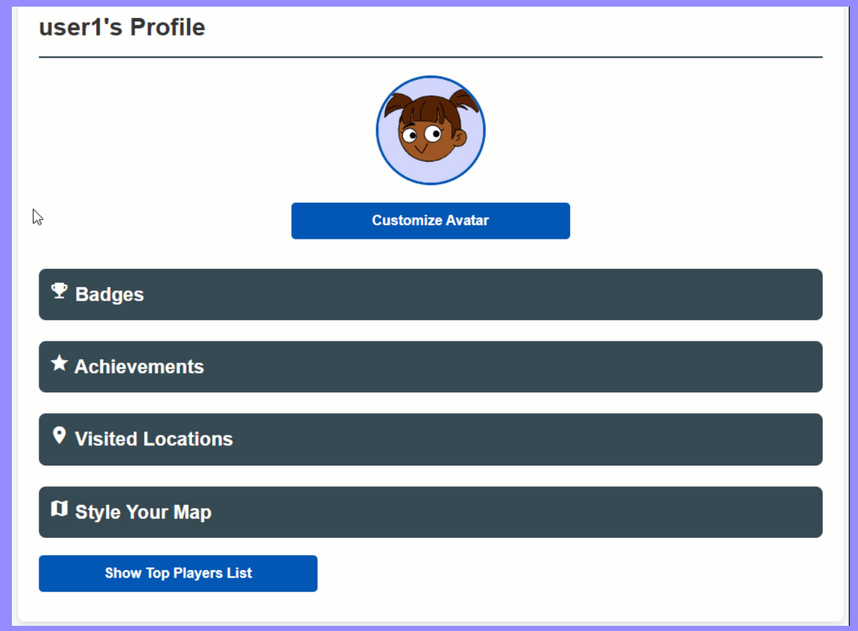
```
const mapDiv = useRef(null);
```

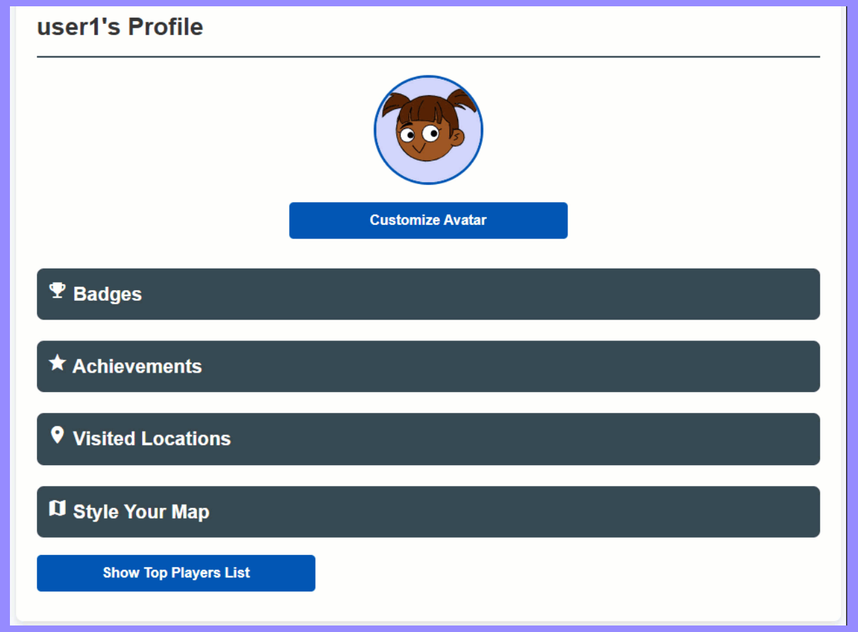
```
useEffect(() => {
  if (!location || !mapDiv.current) return;
  // Map initialization code...
  return () => view.destroy();
}, [location, landmarks, selectedBasemap]);
```



Efficient Rendering – On-Demand Badge Rendering

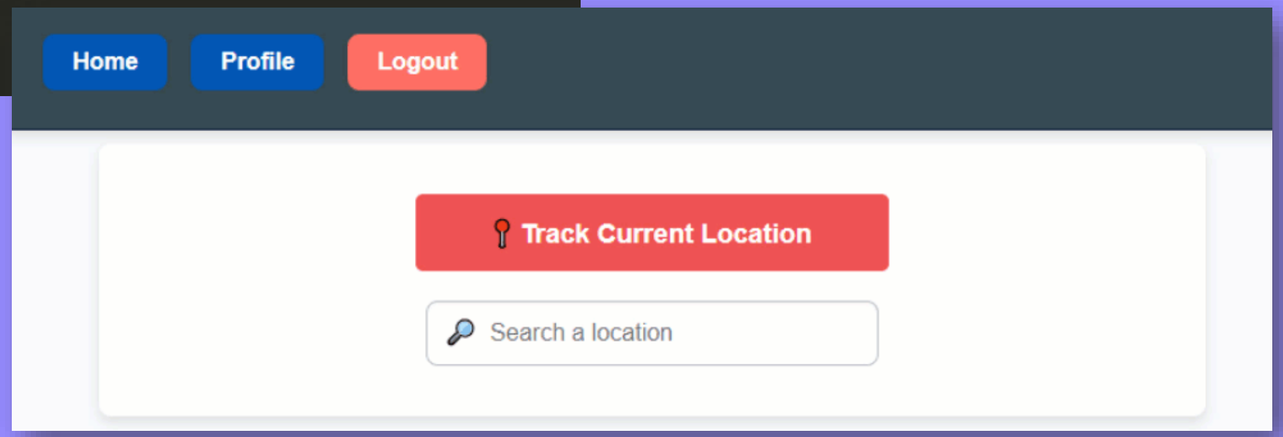
```
{uniqueBadges.length > 0 ? (
    // render badge list and sharing options
  <> ···
  </>
) : (
  <p>No badges earned yet.</p>
)}
```

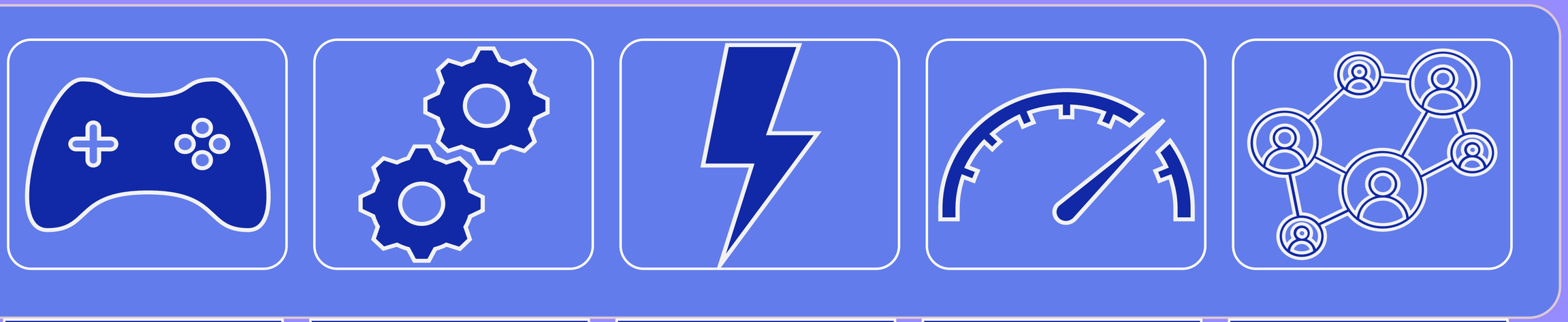




Efficient Rendering – On-Demand Achievement Popups

```
const queueAchievementPopup = (points, text) => {
  setAchievementQueue((prevQueue) => [...prevQueue, { points, text }]);
};

useEffect(() => {
  if (achievementQueue.length > 0 && !showAchievementPopup) {
    const { points, text } = achievementQueue[0];
    showAchievementPopupMessage(points, text);
    setAchievementQueue((prevQueue) => prevQueue.slice(1));
  }
}, [achievementQueue, showAchievementPopup]);
```





G

Gamified UI Elements

A

Advanced State Control

M

Memoization /Modern Optimization

E

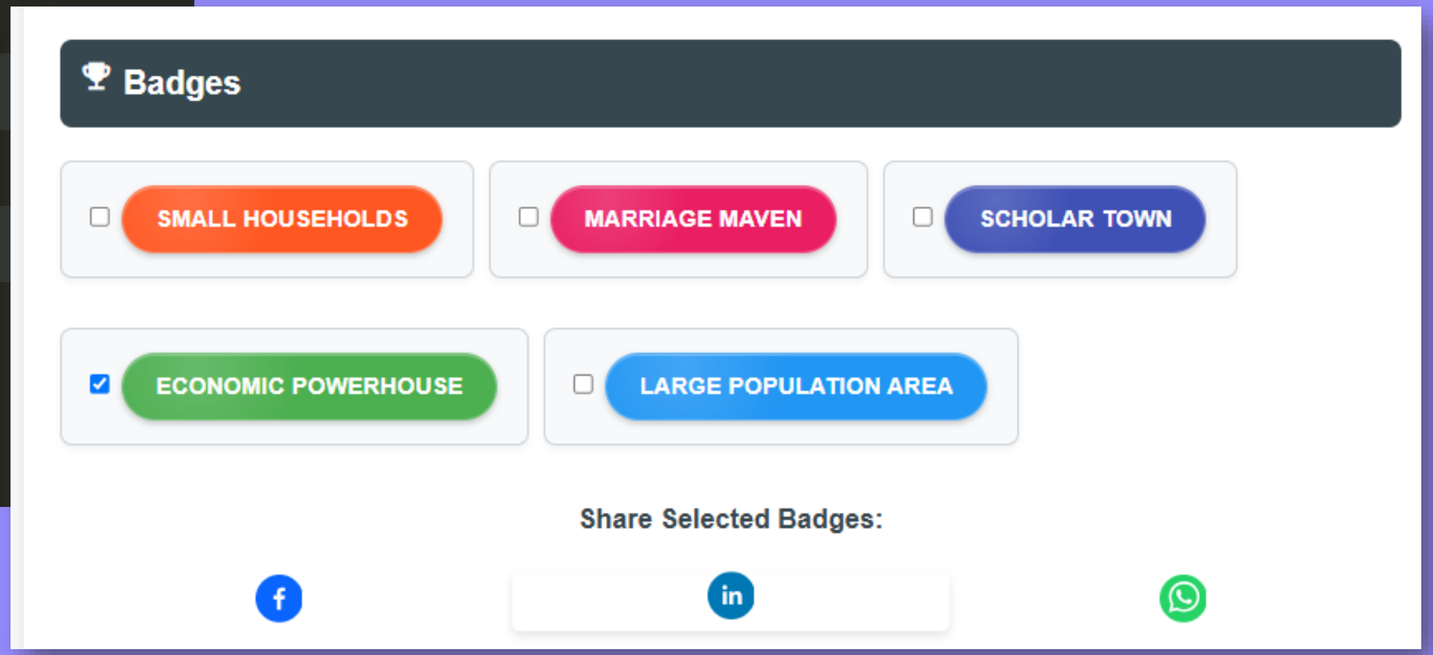
Efficient Rendering

S

Social Interaction

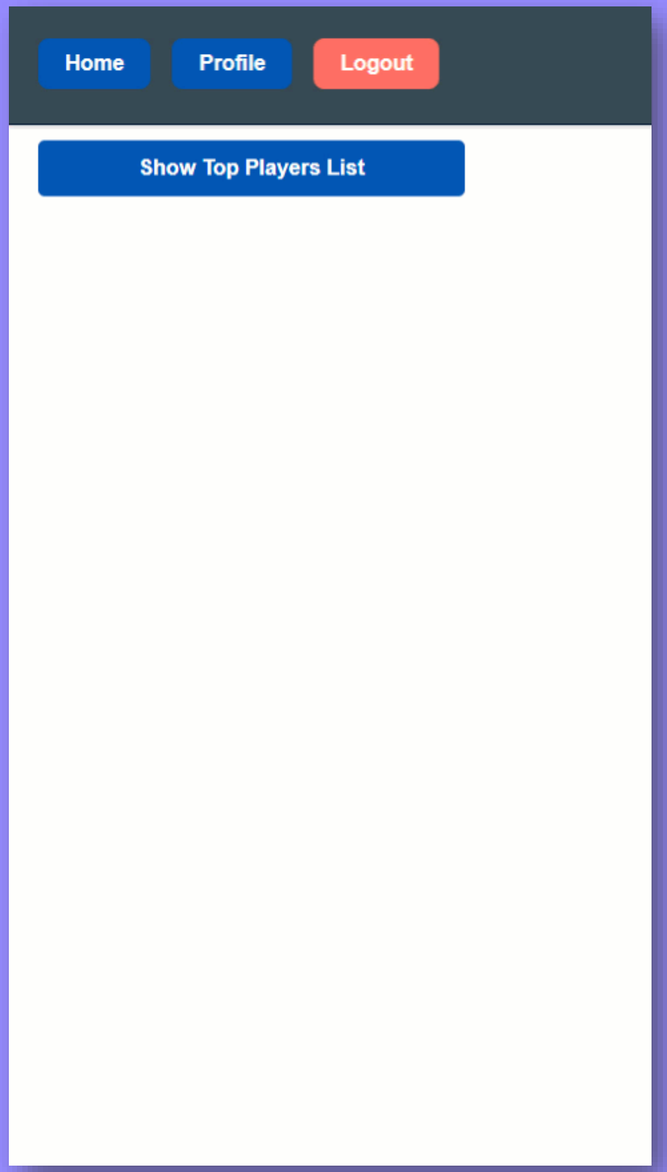
Social Interaction – Share Badges

```
<div className="share-selected">
  <h4>Share Selected Badges:</h4>
  <div className="social-sharing">
    <FacebookShareButton ···
    </FacebookShareButton>
    <LinkedinShareButton ···
    </LinkedinShareButton>
    <WhatsappShareButton ···
    </WhatsappShareButton>
  </div>
</div>
```



Social Interaction – Player List

```jsx
const PlayerList = ({ players }) => {
  const sortedPlayers = useMemo(() => {
    return players.sort((a, b) => b.score - a.score);
  }, [players]);
  return (
    <div className="player-list">
      <h3>Top Players</h3>
      <ul>
        {sortedPlayers.map(player => (
          <Player key={player.id} player={player} />
        ))}
      </ul>
    </div>
  );
};
```



duolingo

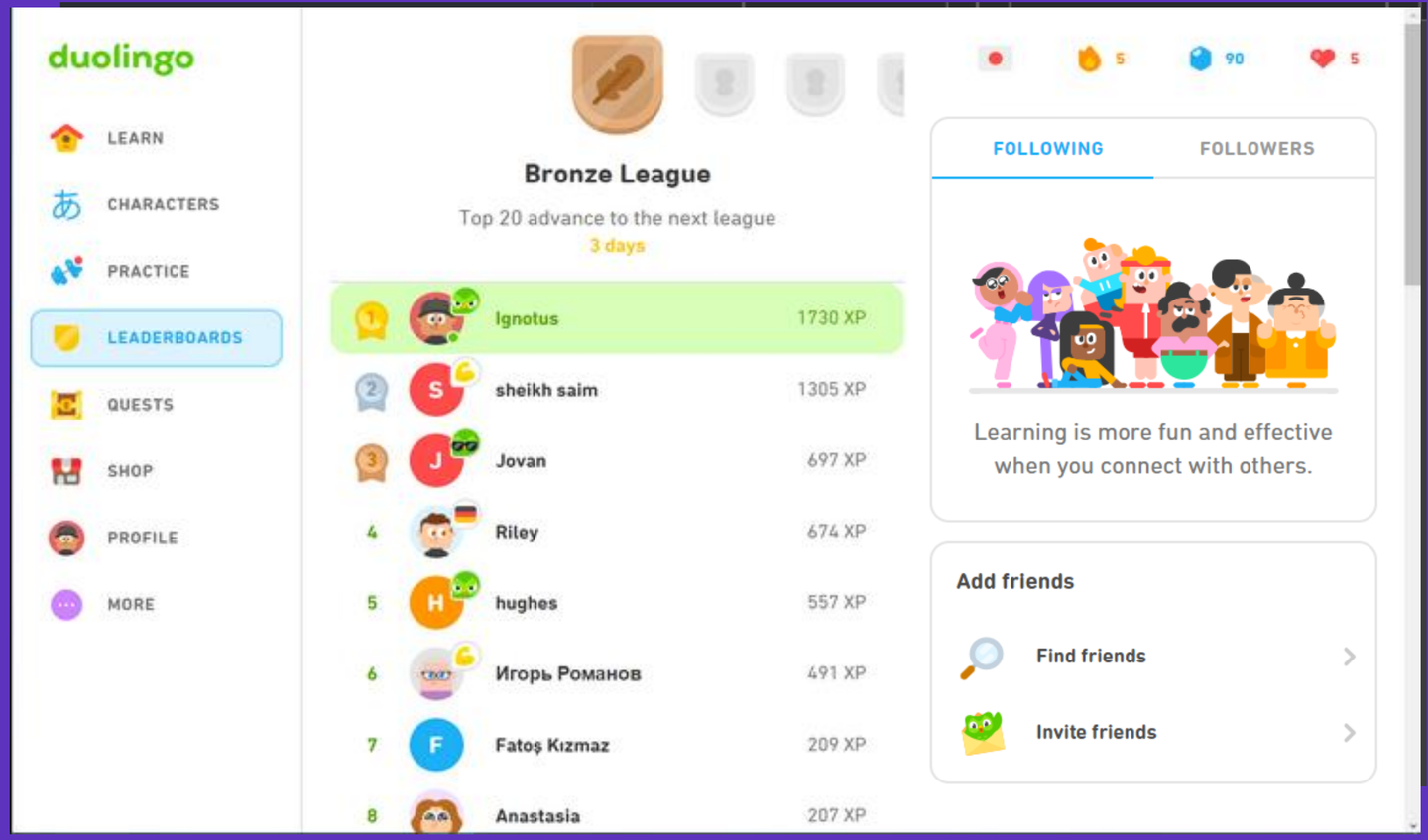
G Streak Counters & XP Bars

A Advanced State Management

M Optimized Lesson Rendering

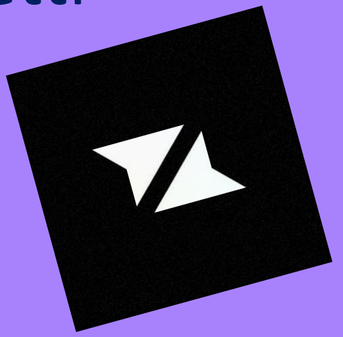
E Efficient Component Rendering

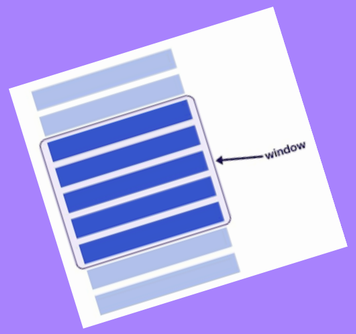
S Leaderboards & Challenges



EXTENSIONS - DROP-IN LIBRARIES

- **G (Feedback & Delight):** framer-motion, canvas-confetti
- **A (Rules & Progress):** Zustand
- **M (Modern Optimization):** react-window
- **E (Efficient Rendering):** why-did-you-render
- **S (Streaks & Social):** Liveblocks






GAMES

Tradeoffs and Takeaways

Good gamification

- Rewards meaningful action
- Clarifies progress
- Creates useful feedback

Bad gamification

- Adds noise
- Rewards empty behavior
- Pressures users
- Hides weak product value

Would this app still matter without the game layer?

THANK YOU,

JS Tek!

Courtney Yatteau

@c_yatteau

courtneyyatteau